

# BÁO CÁO THỰC HÀNH

**Thực hành Nhập môn Mạng máy tính**

**Lab 2: Mật mã học hiện đại**

GVTH: Nguyễn Ngọc Trường

Ngày báo cáo: 2/3/2026

## Nhóm 2 – NT101.Q21.1

### THÔNG TIN CHUNG

| MSSV     | Họ và tên                | Nội dung báo cáo   | % công việc |
|----------|--------------------------|--------------------|-------------|
| 24520946 | Bửu Nguyễn Phúc Vĩnh Lập | Task 2.3, 2.4, 2.5 | 50%         |
| 24521958 | Nguyễn Ngọc Tuyền        | Task 2.1, 2.2, 2.6 | 50%         |

## Nhiệm vụ 2.1

### Ý tưởng:

Bài toán yêu cầu mô phỏng một hệ mật mã Feistel đơn giản để quan sát sự lan truyền khác biệt của dữ liệu qua từng vòng mã hóa. Theo cấu trúc Feistel, ở mỗi vòng ta cập nhật

$$+L_i = R_{i-1}$$

$$+R_i = L_{i-1} \text{ XOR } F(R_{i-1}, K_i)$$

Trong chương trình, bản rõ 8 bit được chia thành hai nửa 4 bit là L và R. Sau đó tiến hành mã hóa 4 vòng với các khóa con sinh ra từ khóa gốc 0x12. Mục tiêu là theo dõi giá trị L, R sau từng vòng đối với hai bản rõ 0xAB và 0xAC, từ đó quan sát mức độ khác biệt tăng dần.

### Thực hiện:

Chương trình định nghĩa hàm  $F(\text{right}, \text{subkey})$  theo scaffold của đề bài, cụ thể trả về giá trị XOR giữa right và subkey, sau đó giới hạn trong 4 bit cuối bằng phép  $\& 0x0F$ .

Tiếp theo, hàm `feistel_round(L_in, R_in, subkey)` được xây dựng theo đúng công thức Feistel:

$$+L\_out = R\_in$$

$$+R\_out = L\_in \wedge F(R\_in, \text{subkey})$$

```
def feistel_round(L_in, R_in, subkey):  
    # LE_i = RE_{i-1}  
    L_out = R_in  
    # RE_i = LE_{i-1} XOR F(RE_{i-1}, K_i)  
    R_out = L_in ^ F(R_in, subkey)  
    return L_out & 0x0F, R_out & 0x0F
```

Hàm `track_avalanche(msg, key)` được dùng để:

1. Tách bản rõ thành hai nửa 4 bit.
2. Tạo 4 khóa con từ khóa gốc.
3. Thực hiện 4 vòng mã hóa.
4. In ra giá trị L, R sau từng vòng.
5. Ghép lại thành bản mã cuối cùng để đối chiếu.

```
def track_avalanche(msg, key):
    # Tách 8 bit thành 2 nửa 4 bit
    L, R = (msg >> 4) & 0x0F, msg & 0x0F
    # Sinh 4 subkey như scaffold
    subkeys = [
        key & 0x0F,
        (key >> 4) & 0x0F,
        (key + 1) & 0x0F,
        (key + 2) & 0x0F
    ]
    print(f"Khởi tạo: L={format(L, '04b')}, R={format(R, '04b')}")
    for i in range(4):
        L, R = feistel_round(L, R, subkeys[i])
        print(f"Vòng {i+1}: L={format(L, '04b')}, R={format(R, '04b')}")
    return (L << 4) | R
```

### Kết quả:

Với khóa 0x12, chương trình thu được kết quả sau:

Đối với M1 = 0xAB

- + Khởi tạo: L=1010, R=1011
- + Vòng 1: L=1011, R=0011
- + Vòng 2: L=0011, R=1001
- + Vòng 3: L=1001, R=1001
- + Vòng 4: L=1001, R=0100
- + Bản mã cuối: 0x94

```
--- Mã hóa M1 ---
Khởi tạo: L=1010, R=1011
Vòng 1: L=1011, R=0011
Vòng 2: L=0011, R=1001
Vòng 3: L=1001, R=1001
Vòng 4: L=1001, R=0100
Bản mã M1: 0x94
```

Đối với M2 = 0xAC

- + Khởi tạo: L=1010, R=1100
- + Vòng 1: L=1100, R=0100
- + Vòng 2: L=0100, R=1001
- + Vòng 3: L=1001, R=1110
- + Vòng 4: L=1110, R=0011

+ Bản mã cuối: 0xE3

```

--- Mã hóa M2 ---
Khởi tạo: L=1010, R=1100
Vòng 1: L=1100, R=0100
Vòng 2: L=0100, R=1001
Vòng 3: L=1001, R=1110
Vòng 4: L=1110, R=0011
Bản mã M2: 0xE3

```

### Nhận xét:

Kết quả cho thấy chỉ cần thay đổi nhỏ ở bản rõ đầu vào thì qua nhiều vòng Feistel, trạng thái L, R sẽ thay đổi ngày càng rõ rệt. Điều này thể hiện cơ chế khuếch tán thông tin trong cấu trúc Feistel: sai khác ban đầu được lan truyền dần qua từng vòng mã hóa. Đây cũng là cơ sở giúp các hệ mật mã như DES đạt được độ an toàn cao hơn so với các phép biến đổi đơn giản

## Nhiệm vụ 2.2

### Ý tưởng:

Bài này nhằm so sánh sự khác nhau giữa các mode hoạt động của AES khi mã hóa dữ liệu có tính lặp. Chuỗi được dùng là

"UIT\_LAB\_UIT\_LAB\_UIT\_LAB\_UIT\_LAB\_", có độ dài 32 byte nên chia thành đúng 2 block AES. Do hai block bản rõ giống nhau, em dùng trường hợp này để kiểm tra xem ở mỗi mode thì bản mã của hai block có giống nhau hay không. Theo yêu cầu của đề, em thực hiện trước với hai mode là ECB và CBC, sau đó thử thêm CFB, OFB và CTR để quan sát rõ hơn sự khác biệt.

Thực hiện:

Em sử dụng thư viện pycryptodome và khởi tạo khóa b'1234567890123456' đúng như scaffold. Với mode CBC, em thêm một IV cố định dài 16 byte.

```

from Crypto.Cipher import AES

key = b'1234567890123456'
iv = b'ABCDEF1234567890' # IV cố định, đúng 16 bytes
plaintext = b"UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_" # 32 bytes = 2 block AES

```

Em mã hóa plaintext bằng AES-ECB rồi in bản mã dưới dạng hexadecimal. Sau đó em tách ciphertext thành từng block 16 byte để so sánh. Tiếp theo, em làm tương tự với AES-CBC.

Sau khi hoàn thành hai mode chính theo yêu cầu đề bài, em thử thêm các mode

CFB, OFB và CTR để quan sát khi dữ liệu đầu vào bị lặp thì các mode này biểu hiện ra sao ở các block đầu ra.

```
# 1) AES-ECB
cipher_ecb = AES.new(key, AES.MODE_ECB)
ct_ecb = cipher_ecb.encrypt(plaintext)

# 2) AES-CBC
cipher_cbc = AES.new(key, AES.MODE_CBC, iv)
ct_cbc = cipher_cbc.encrypt(plaintext)

print("PLAINTEXT BLOCKS:")
print_blocks("Plaintext", plaintext)

print(f"\nECB: {ct_ecb.hex()}")
print_blocks("ECB blocks", ct_ecb)

print(f"\nCBC: {ct_cbc.hex()}")
print_blocks("CBC blocks", ct_cbc)

# So sánh nhanh
print("\nSO SÁNH:")
print("ECB block 1 == block 2 ?", ct_ecb[:16] == ct_ecb[16:32])
print("CBC block 1 == block 2 ?", ct_cbc[:16] == ct_cbc[16:32])

# 3) Thử thêm các mode khác trong lab
cipher_cfb = AES.new(key, AES.MODE_CFB, iv=iv, segment_size=128)
ct_cfb = cipher_cfb.encrypt(plaintext)

cipher_ofb = AES.new(key, AES.MODE_OFB, iv=iv)
ct_ofb = cipher_ofb.encrypt(plaintext)

cipher_ctr = AES.new(key, AES.MODE_CTR, nonce=b'12345678')
ct_ctr = cipher_ctr.encrypt(plaintext)

print(f"\nCFB: {ct_cfb.hex()}")
print_blocks("CFB blocks", ct_cfb)

print(f"\nOFB: {ct_ofb.hex()}")
print_blocks("OFB blocks", ct_ofb)

print(f"\nCTR: {ct_ctr.hex()}")
print_blocks("CTR blocks", ct_ctr)
```

**Kết quả:**

## 1. Plaintext

- Block 1: 5549545f4c41425f5549545f4c41425f
- Block 2: 5549545f4c41425f5549545f4c41425f

```
Block 1: 02b0647de210da452e3bdcadc415814e
Block 2: 02b0647de210da452e3bdcadc415814e
```

Kết quả này cho thấy 2 block bản rõ là giống nhau hoàn toàn.

## 2. AES-ECB

- Ciphertext:  
02b0647de210da452e3bdcadc415814e02b0647de210da452e3bdcadc415814e
- ECB Block 1:  
02b0647de210da452e3bdcadc415814e
- ECB Block 2:  
02b0647de210da452e3bdcadc415814e
- So sánh:  
ECB block 1 == block 2 ? True

```
ECB: 02b0647de210da452e3bdcadc415814e02b0647de210da452e3bdcadc415814e
```

```
ECB blocks
```

```
Block 1: 02b0647de210da452e3bdcadc415814e
```

```
Block 2: 02b0647de210da452e3bdcadc415814e
```

```
CBC: 649df7e306f27e360811895f892cddfcf345482e61b3486371d46d0b8372fe1b
```

```
CBC blocks
```

```
Block 1: 649df7e306f27e360811895f892cddfc
```

```
Block 2: f345482e61b3486371d46d0b8372fe1b
```

```
SO SÁNH:
```

```
ECB block 1 == block 2 ? True
```

```
CBC block 1 == block 2 ? False
```

## 3. AES-CBC

- Ciphertext:  
649df7e306f27e360811895f892cddfcf345482e61b3486371d46d0b8372fe1b

- CBC Block 1:  
649df7e306f27e360811895f892cddfc
- CBC Block 2:  
f345482e61b3486371d46d0b8372fe1b
- So sánh:  
CBC block 1 == block 2 ? False

```
CBC: 649df7e306f27e360811895f892cddfcf345482e61b3486371d46d0b8372fe1b
```

```
CBC blocks
```

```
Block 1: 649df7e306f27e360811895f892cddfc
```

```
Block 2: f345482e61b3486371d46d0b8372fe1b
```

```
SO SÁNH:
```

```
ECB block 1 == block 2 ? True
```

```
CBC block 1 == block 2 ? False
```

#### 4. AES-CFB

- Ciphertext:  
a0e62dd238ba66fde5519b5f7835d68531da824f60a94f40cf8bf33b0fd21664
- CFB Block 1:  
a0e62dd238ba66fde5519b5f7835d685
- CFB Block 2:  
31da824f60a94f40cf8bf33b0fd21664

```
CFB: a0e62dd238ba66fde5519b5f7835d68531da824f60a94f40cf8bf33b0fd21664
```

```
CFB blocks
```

```
Block 1: a0e62dd238ba66fde5519b5f7835d685
```

```
Block 2: 31da824f60a94f40cf8bf33b0fd21664
```

#### 5. AES-OFB

- Ciphertext:  
a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
- OFB Block 1:  
a0e62dd238ba66fde5519b5f7835d685
- OFB Block 2:  
5e9136fbba3633fb61799a79134b5b05



```
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
```

```
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
```

```
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
OFB: a0e62dd238ba66fde5519b5f7835d6855e9136fbba3633fb61799a79134b5b05
```

```
OFB blocks
```

```
Block 1: a0e62dd238ba66fde5519b5f7835d685
```

```
Block 2: 5e9136fbba3633fb61799a79134b5b05
```

## 6. AES-CTR

- Ciphertext:  
69c911a07a43abae647828cb9d7c5d392de106e56197b52142cdcf917f0836ff
- CTR Block 1:  
69c911a07a43abae647828cb9d7c5d39
- CTR Block 2:  
2de106e56197b52142cdcf917f0836ff

```
CTR: 69c911a07a43abae647828cb9d7c5d392de106e56197b52142cdcf917f0836ff
```

```
CTR blocks
```

```
Block 1: 69c911a07a43abae647828cb9d7c5d39
```

```
Block 2: 2de106e56197b52142cdcf917f0836ff
```

### Nhận xét:

Ở mode ECB, do mỗi block được mã hóa riêng nên hai block bản rõ giống nhau sẽ cho ra hai block bản mã giống nhau. Vì vậy mode này làm lộ cấu trúc dữ liệu nếu đầu vào có tính lặp.

Ở mode CBC, block sau phụ thuộc vào block trước và IV nên dù dữ liệu đầu vào lặp thì bản mã vẫn khác nhau. Khi thử thêm CFB, OFB và CTR, em cũng thấy các mode này che giấu lặp tốt hơn ECB. Từ đó có thể kết luận ECB không phù hợp để mã hóa dữ liệu nhiều khối có cấu trúc lặp.



## Nhiệm vụ 2.3

### Ý tưởng:

Avalanche effect là khi có thay đổi nhỏ ở đầu vào sẽ dẫn đến kết quả thay đổi lớn ở cuối giống như khi núi tuyết trượt.

Ý tưởng:

- Ở đây khi mã hóa DES chỉ cần 1 thay đổi ở ký tự đầu vào mà vẫn giữ nguyên khóa thì ciphertext đầu ra thay đổi như thế nào
- Mã hóa 2 chuỗi là “STAYHOME” và “STAYHOMA”
- Test bằng 3 key là key ở đề và 2 key là mã số sinh viên
- Kết quả sẽ đếm và tính toán tỉ lệ thay đổi

### Thực hiện:

```
from Crypto.Cipher import DES

p1 = b'STAYHOME'
p2 = b'STAYHOMA'

def avalanche_test(key):
    cipher = DES.new(key, DES.MODE_ECB)
    c1 = cipher.encrypt(p1)
    c2 = cipher.encrypt(p2)
    bin_c1 = bin(int.from_bytes(c1, 'big'))[2:].zfill(64) # [2:].zfill(64) không có cái này in ra số 0 ở trước bị lỗi gg
    bin_c2 = bin(int.from_bytes(c2, 'big'))[2:].zfill(64) # dùng để fill vào
    hamming_distance = 0
    for bit1, bit2 in zip(bin_c1, bin_c2):
        if bit1 != bit2:
            hamming_distance += 1
    TongBits = 64
    PhanTram = (hamming_distance / TongBits) * 100
    print(f"Test với key: {key}")
    print(f"Ma 1 (STAYHOME): {bin_c1}")
    print(f"Ma 2 (STAYHOMA): {bin_c2}")
    print(f"So bit khác nhau (Hamming Distance): {hamming_distance} / {TongBits}")
    print(f"Ty lệ thay đổi: {PhanTram:.2f}%\n")

keys = [b'87654321', b'24520946', b'24521958']
for key in keys:
    avalanche_test(key)
```

- Hàm avalanche\_test nhận vào một chìa khóa (key), dùng nó để mã hóa cả p1 và p2 thành 2 bản mã (c1, c2).
- Đổi sang nhị phân, chuyển c1 và c2 thành hai chuỗi toàn số 0 và 1, đồng thời căn chỉnh cho đủ đúng 64 bit (zfill(64)).
- Đếm lỗi (Hamming Distance): Đặt 2 chuỗi nhị phân cạnh nhau, vòng lặp for sẽ dò từng vị trí xem có bao nhiêu cặp bit khác biệt nhau thì cộng dồn vào biến hamming\_distance.
- Tính toán và In kết quả. Lấy số bit khác biệt chia cho tổng 64 bit để ra tỷ lệ phần trăm (%), sau đó in ra màn hình.
- Vòng lặp cuối cùng sẽ tự động gọi hàm trên chạy 3 lần với 3 chìa khóa khác nhau.

**Kết quả:**

```

C:\Windows\system32\cmd.exe
Test với key: b'87654321'
Ma 1 (STAYHOME): 1010000111000100110000010011000101011111100001100000010100110101
Ma 2 (STAYHOMA): 1110111000010011111010110000011000110000000111111001011010111110
Số bit khác nhau (Hamming Distance): 37 / 64
Tỷ lệ thay đổi: 57.81%

Test với key: b'24520946'
Ma 1 (STAYHOME): 1000100110100010011000111001100001101101111011101111010101101010
Ma 2 (STAYHOMA): 1111010101001101010101010010010100010000100100100011110111011111
Số bit khác nhau (Hamming Distance): 41 / 64
Tỷ lệ thay đổi: 64.06%

Test với key: b'24521958'
Ma 1 (STAYHOME): 0000000110110110110011011001110011100000010110101011110010000011
Ma 2 (STAYHOMA): 11010100100110000101100101100000001010011110110110110001111010
Số bit khác nhau (Hamming Distance): 37 / 64
Tỷ lệ thay đổi: 57.81%

Press any key to continue . . .

```

**Nhận xét:**

Cả sự thay đổi khóa và sự thay đổi dữ liệu điều mạng tới ảnh hưởng lớn tới kết quả cipher text đưa ra các chuỗi khác nhau hoàn toàn chứng tỏ lý thuyết avalanche effect là đúng.

**Nhiệm vụ 2.4****Ý tưởng:**

1. Tạo dữ liệu 1000 byte.
2. Mã hóa dữ liệu bằng AES-128.
3. Làm hỏng bản mã bằng cách đảo 1 bit tại byte thứ 26.
4. Giải mã bản mã đã bị lỗi và quan sát có bao nhiêu khối dữ liệu bản rõ bị hỏng với chế độ mã hóa được sử dụng lần lượt là ECB, CBC, CFB, hoặc OFB.

**Thực hiện:**

```

✓ from Crypto.Cipher import AES
✓ from Crypto.Util.Padding import pad
import os
padded_data = pad(b'1234567890' * 1000, 16)
key = os.urandom(16)
iv = os.urandom(16)
✓ def check(ten_che_do, mode):
✓     if ten_che_do == 'ECB':
✓         cipher_enc = AES.new(key, mode)
✓     elif ten_che_do == 'CBC':
✓         cipher_enc = AES.new(key, mode, iv)
✓     elif ten_che_do == 'CFB':
✓         cipher_enc = AES.new(key, mode, iv)
✓     elif ten_che_do == 'OFB':
✓         cipher_enc = AES.new(key, mode, iv)

        ban_ma = bytearray(cipher_enc.encrypt(padded_data))

```

Chuỗi ký tự 1234567890 lặp 100 lần để đủ 1000 byte

Hàm check được dùng để khởi tạo 4 trường hợp, bởi vì ECB không dùng iv (Initialization Vector (Vector khởi tạo)) nên phải xét riêng trường hợp

Chuỗi sau đó sẽ được đưa vào bytearray() để biến thành dạng mảng chỉnh sửa được

```

        ban_ma[25] ^= 0x01
✓ if ten_che_do == 'ECB':
✓     cipher_dec = AES.new(key, mode)
✓ elif ten_che_do == 'CBC':
✓     cipher_dec = AES.new(key, mode, iv)
✓ elif ten_che_do == 'CFB':
✓     cipher_dec = AES.new(key, mode, iv)
✓ elif ten_che_do == 'OFB':
✓     cipher_dec = AES.new(key, mode, iv)

        ban_loi = cipher_dec.decrypt(bytes(ban_ma))

        so_byte_loi = sum(1 for b1, b2 in zip(padded_data, ban_loi) if b1 != b2)
        print(f"{ten_che_do} {so_byte_loi} byte loi")

check('ECB', AES.MODE_ECB)
check('CBC', AES.MODE_CBC)
check('CFB', AES.MODE_CFB)
check('OFB', AES.MODE_OFB)

```

- Tạo 1 byte lỗi ở vị trí số 26 bằng cách lật ngược 1 bit của byte đó
- Khởi if else để giải mã
- Sau khi giải mã tiến hành so sánh đếm số byte lỗi, dùng lệnh zip() so sánh dữ liệu ban đầu với dữ liệu giải mã văn bản lỗi, rồi dùng hàm sum tính số byte lỗi.

**Kết quả:**

```

C:\Windows\system32\cmd.exe
ECB 16 byte loi
CBC 17 byte loi
CFB 17 byte loi
OFB 1 byte loi
Press any key to continue . . .

```

**Nhiệm vụ 2.5**

| Tiêu chí        | DES          | 3DES                 | AES                    |
|-----------------|--------------|----------------------|------------------------|
| Năm ra đời      | 1977         | 1998                 | 2001                   |
| Kích thước Khóa | 56 bit       | 112 bit hoặc 168 bit | 128, 192, hoặc 256 bit |
| Kích thước Khối | 64 bit       | 64 bit               | 128 bit                |
| Tốc độ mã hóa   | Rất Chậm     | Chậm                 | Nhanh                  |
| Độ an toàn      | Đã bị phá vỡ | Đang bị loại bỏ      | Cực kỳ an toàn         |

Tại sao không sử dụng 2DES

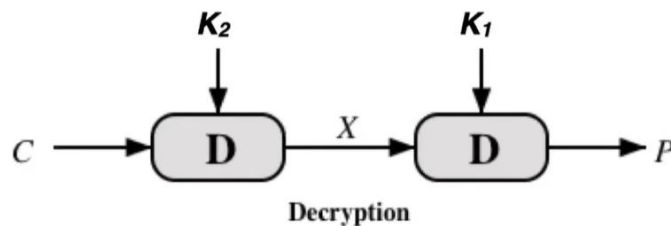
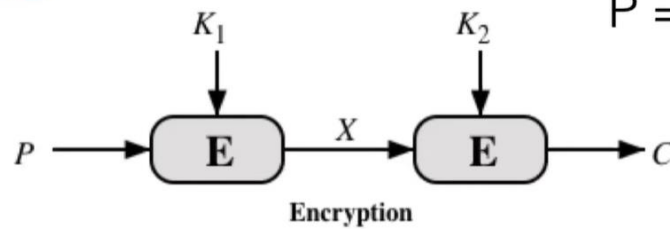
Theo tư duy thông thường: Nếu 1 vòng DES (khóa 56 bit) không an toàn, ta mã hóa 2 vòng (2DES) với 2 khóa khác nhau. Thế nhưng, giới hacker đã tìm ra một lỗ hổng chí mạng gọi là Tấn công gặp nhau ở giữa (Meet-in-the-Middle Attack).

Cách lỗ hổng này hoạt động:

## 2DES

$$C = E_{K_2} ( E_{K_1}(P) )$$

$$P = D_{K_1} ( D_{K_2}(C) )$$

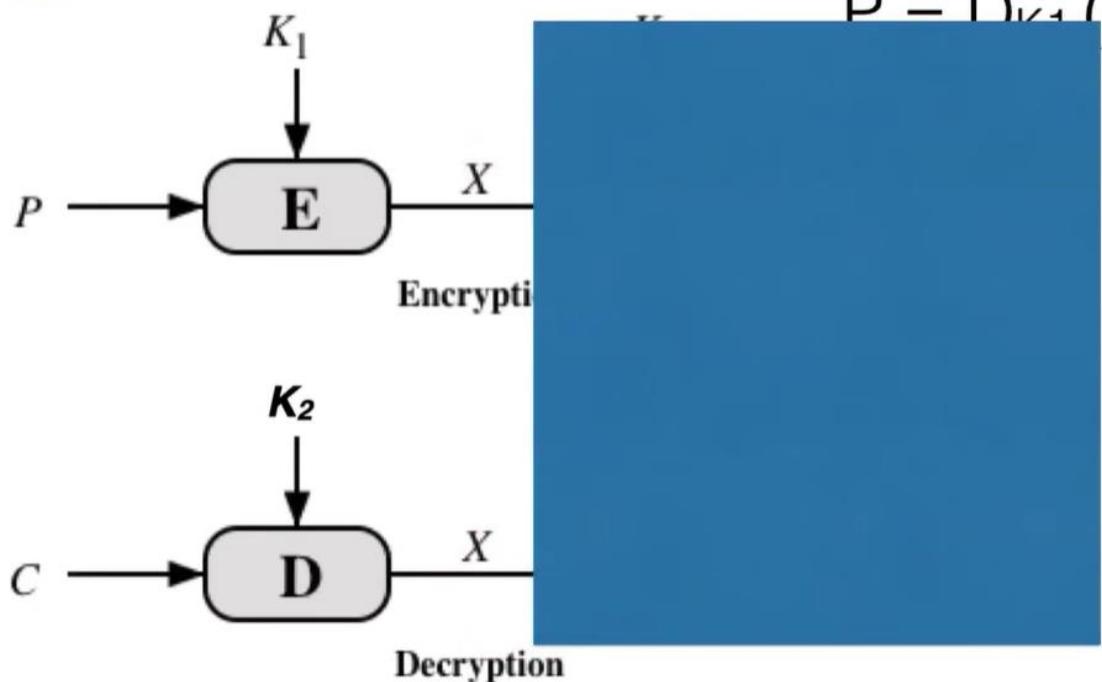


Hình 1

## 2DES

$$C = E_{K_2} ($$

$$P = D_{K_1} ($$



Hình 2

Thì chúng ta có thể nhận thấy theo hình rằng viết dùng plain text để mã hóa bằng khóa  $K_1$  thì cho ra được  $X$ , dùng ciphertext và dùng khóa  $K_2$  để giải mã thì cũng ra được  $X$

Có nghĩa là để tấn công thì chúng ta phải 1 phần nào đó biết về plain text ban đầu sau đó cùng lúc brute force thử tất cả các trường hợp mã hóa đối với plain text  $2^{56}$  và làm tương tự nhưng là giải mã với cipher text cũng là  $2^{56}$

Sau đó thì đem so sánh X cái nào giống nhau thì có nghĩa là tìm được khóa K1 và khóa K2

**Hậu quả:**

Thay vì phải thử  $2^{112}$  phép tính như kỳ vọng, kẻ tấn công chỉ cần làm  $2^{56} + 2^{56} = 2^{57}$  phép tính.

**Kết luận:** 2DES hoàn toàn là một sự lãng phí. Nó làm máy tính của bạn chạy chậm đi gấp đôi (vì phải mã hóa 2 lần), nhưng độ an toàn mang lại thì gần như giống DES. Đó là lý do người ta phải nhảy thẳng lên 3DES chứ không ai dùng 2DES cả.

## Nhiệm vụ 2.6

**Ý tưởng:**

Bài này ôn lại các kiến thức nền về số nguyên tố, GCD và phép lũy thừa modulo lớn. Đây là các nội dung quan trọng trong mật mã khóa công khai, đặc biệt là RSA.

Chương trình em làm gồm ba phần chính:

1. kiểm tra và sinh số nguyên tố,
2. tính ước chung lớn nhất của hai số lớn,
3. tính lũy thừa modulo với số mũ lớn.

Thực hiện:

Để kiểm tra số nguyên tố, em cài đặt thuật toán Miller–Rabin vì cách này nhanh hơn nhiều so với thử chia tuần tự khi làm việc với số lớn. Từ đó em viết thêm hàm sinh số nguyên tố ngẫu nhiên theo số bit yêu cầu như 8 bit, 16 bit và 64 bit.

```
# Kiểm tra số nguyên tố bằng Miller-Rabin
def kiem_tra_nguyen_to(n, so_lan_kiem_tra=10):
    if n < 2:
        return False
    cac_so_nguyen_to_nho = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    for p in cac_so_nguyen_to_nho:
        if n == p:
            return True
        if n % p == 0:
            return False
    r = 0
    d = n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(so_lan_kiem_tra):
        a = secrets.randbelow(n - 3) + 2
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True
```

Tiếp theo, em viết hàm tìm số nguyên tố liền trước để lấy ra 10 số nguyên tố lớn nhất nhỏ hơn  $2^{89} - 1$ .

```
# Sinh số nguyên tố có số bit cho trước
def sinh_so_nguyen_to(so_bit):
    while True:
        n = secrets.randbits(so_bit)
        n |= (1 << (so_bit - 1)) # đảm bảo đúng số bit
        n |= 1 # đảm bảo là số lẻ
        if kiem_tra_nguyen_to(n):
            return n
```

Ở phần GCD, em dùng thuật toán Euclid, lặp phép chia lấy dư cho đến khi số dư bằng 0.

```
# Tìm ước chung lớn nhất bằng Euclid
def uoc_chung_lon_nhat(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

Ở phần cuối, em dùng phương pháp bình phương và nhân để tính  $a^x \bmod p$ , nhờ vậy chương trình vẫn chạy nhanh khi số mũ lớn hơn 40.



```
# Tính lũy thừa modulo bằng phương pháp bình phương và nhân
def luy_thua_modulo(a, x, p):
    ket_qua = 1
    a = a % p
    while x > 0:
        if x % 2 == 1:
            ket_qua = (ket_qua * a) % p
        a = (a * a) % p
        x //= 2
    return ket_qua

# Tìm số nguyên tố liền trước một số n
def nguyen_to_lien_truoc(n):
    ung_vien = n - 1
    if ung_vien % 2 == 0:
        ung_vien -= 1
    while ung_vien > 2:
        if kiem_tra_nguyen_to(ung_vien):
            return ung_vien
        ung_vien -= 2
    return 2
```

**Kết quả:**

Số nguyên tố 8 bit: 233

Số nguyên tố 16 bit: 40357

Số nguyên tố 64 bit: 10540414889309291071

10 số nguyên tố lớn nhất nhỏ hơn  $2^{89} - 1$ :

1. 618970019642690137449562091
2. 618970019642690137449562081
3. 618970019642690137449562063
4. 618970019642690137449562043
5. 618970019642690137449562013
6. 618970019642690137449562009
7. 618970019642690137449561847
8. 618970019642690137449561791
9. 618970019642690137449561671
10. 618970019642690137449561509

Kiểm tra số 9999999967: kết quả là True, tức là số nguyên tố.

UCLN của 123456789123456789123456789 và  
987654321987654321987654321 là:  
9000000009000000009

Kết quả phép tính  $7^{40} \bmod 19$  là: 7

10 số nguyên tố lớn nhất nhỏ hơn  $2^{89} - 1$ :

1. 618970019642690137449562091
2. 618970019642690137449562081
3. 618970019642690137449562063
4. 618970019642690137449562043
5. 618970019642690137449562013
6. 618970019642690137449562009
7. 618970019642690137449561847
8. 618970019642690137449561791
9. 618970019642690137449561671
10. 618970019642690137449561509

9999999967 là số nguyên tố? True

UCLN = 9000000009000000009

$7^{40} \bmod 19 = 7$

#### Nhận xét:

Qua bài này có thể thấy các thuật toán trong lý thuyết số rất quan trọng trong mật mã học. Miller–Rabin giúp kiểm tra số nguyên tố nhanh hơn nhiều so với cách làm đơn giản. Thuật toán Euclid tính GCD rất hiệu quả ngay cả khi hai số rất lớn. Phương pháp bình phương và nhân cũng giúp tính lũy thừa modulo nhanh và phù hợp với các bài toán mật mã thực tế.

Nhìn chung, đây là phần nền tảng để hiểu cách tạo khóa và tính toán trong RSA.

**HẾT**